

Spring 2026 CS 4120

From Text to Time: An NLP Approach to Recipe Duration Prediction

Jennifer Cui, Ethan Nguyen, Hannah Piersol, Arshia Verma

I. Introduction

As a member of the American working class, you are finally home after a long day at work, and despite your fatigue, you have one more task to complete: *cooking*. Before you're able to retire for the day, you have to factor in the time it takes to prepare your ingredients, cook your meal, wash the dishes and suddenly, your time to rest before your next day of work gradually slips away from you. Fortunately for you, finding recipes online with an estimated time of completion is more common than not, however, every recipe for the same dish reports a different predicted duration. How can you know which is most accurate? Additionally, you may not have all of the ingredients called for in a recipe; how does that affect the total preparation time?

For many working Americans, fitting time to prepare and cook a filling, healthy dinner into their busy, delicate schedules can be quite daunting. According to a study on the average time spent on home food preparation from the School of Public Health, individuals who spent less time on food preparation typically prioritized convenience, spent significantly more money on food away from home compared to those who spent more time on food preparation, and showed a lower intake of fruits and vegetables (Monsivais, Aggarwal, Drewnowski). Many Americans, however, may not be able to dedicate that hour due to daily circumstances such as multiple jobs, family obligations, and access to the nearest grocery store. Although this pattern presents itself, cooking healthy meals does not need to correlate to long meal preparation times, and smart planning can lead to meeting dietary recommendations while fitting into the schedules of the working American.

Resources such as recipes with estimated time of completion are great ways to plan around a busy schedule, however, these predictions often vary recipe to recipe. The purpose of this study is to build a natural language processing model that can learn from structured recipe data and predict how long a meal will take when given a corpus of ingredients and/or directions. The motivation is to help reduce the burden of long cooking times and encourage others to partake in frequent meal preparation by providing an estimated completion time using a holistic approach that synthesizes and learns durations from a wide variety of recipes by basing predictions on how factors such as ingredients, cooking techniques, and quantity of ingredients have presented and influenced cooking times in the dataset with the hope that the model will return more accurate time estimates that can help individuals approach cooking more confidently.

II. Methodology

Data

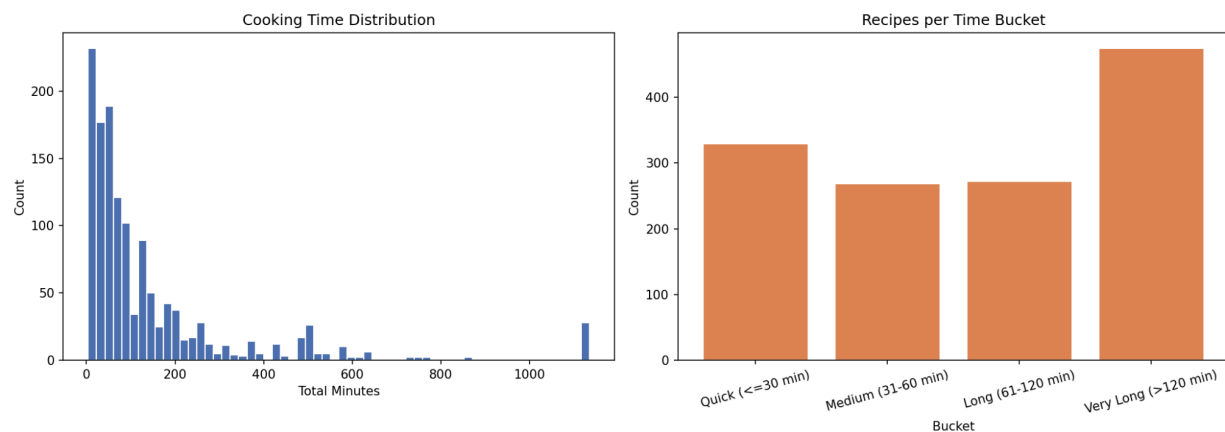
The textual data is sourced from Kaggle's [Recipe Dataset](#) and [Food.com Recipes and Interactions](#), combined containing over 200,000 recipes broken down by recipe name, total time, servings, ingredients, and directions. The corpus consists of recipe documents represented as

concatenated text from the ingredient and direction fields. Total cooking time is used as the target label, while ingredients and directions serve as model inputs. Additional meta data such as recipe rating may be used as a reliability signal.

Originally, the model was built solely on the Kaggle Recipe Dataset which contains over 1,000 American recipes. After our first iteration of testing, it was clear that there was an underrepresentation of recipes that fit the “Very Long” qualification which was reflected in our model’s poor performance.

To ensure that the model had adequate representation of recipes in the “Very Long” range, the Recipe Dataset was augmented with recipes with longer estimated times of completion from the Food.com Recipes and Interactions dataset. This approach introduced a slight overrepresentation of “Very Long” recipes as shown in the histogram shown in *Figure 1*. After augmentation, the time distribution still shows a right skew despite a higher count of “Very Long” recipes which could suggest that our model predicts “Very Long” more often than it should. After preprocessing, the working dataset contains 1,390 recipes distributed across four time buckets: Quick (≤ 30 min), Medium (31-60 min), Long (61-120 min), and Very Long (>120 min), with approximately 316, 263, 267, and 470 recipes respectively.

Figure 1
Cooking Time Diversity after Augmentation



Preprocessing

Raw recipe data requires heavy preprocessing before feeding it into a model for training. Human language can be difficult for a language learning model to understand, for example, features such as quantity. Our primary preprocessing focus is cleaning and structuring ingredients and instruction text to reduce numerical ambiguity.

Raw text is initially parsed and normalized using the Python libraries `ingredient_parser_nlp` and `pint` to break down raw ingredient strings into structured components. Normalization with the `pint` library ensures that the quantity of ingredients are on comparable scales allowing the model to distinguish time differences between amounts of ingredients. Special characters and formatting artifacts are then removed, and instruction text is normalized by removing newline

characters and excess whitespace. Explicit time expressions are also removed from instructional text to prevent any “cheating”, ensuring model predictions are informed by semantic context.

The corpus is then constructed by concatenating the cleaned ingredient and cleaned instructional text together and fitting a TF-IDF vectorizer while L2-normalizing the resulting matrix. Concurrently, our vectors are embedded with rich, culinary context using RecipeBERT, a fine tuned BERT model that has been trained on over one million recipes (Merreddy, Beedareddy). The transformer produces dense embeddings that encode each ingredient or direction with important contextual information that helps the model understand similarity between culinary terms that TF-IDF may not be able to capture such as the similarity between “saute” and “pan-fry”. The resulting matrix is then horizontally stacked with the TF-IDF matrix and normalized scalar features.

III. Models, Experimentation, and Results

Experimentation explored the recipe estimated time challenge framed as both a regression and classification task with Linear Regression and Logistic Regression models respectively, each trained on an 80-20 train-test split of the data. Each model is implemented using models from the ScikitLearn Linear Models library.

The Linear Regression model explores the value in predicting exact cook time of a recipe in minutes. The model was evaluated by looking at metrics Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 score. With an MAE of 176.7 minutes and an RMSE of 313.0 minutes, it was clear that the Linear Regression model struggled to capture the patterns and trends in the recipe dataset with a simple linear mapping. This is especially highlighted in the R^2 score of -0.0303 which indicates that the model performs *worse* than simply predicting the mean cook time for every recipe. Fitting a linear relationship actively worsens predictions, confirming that the underlying patterns in recipe data are nonlinear.

In comparison, the Multiclass Logistic Regression model intended to predict a time range as opposed to the direct time. Using the same train-test split, the Logistic Model was evaluated on its accuracy, precision, recall, and weighted and macro F1-score. The results are as follows:

Figure 2

Multiclass Logistic Regression Results by Class

	precision	recall	f1-score	support
Quick (<=30 min)	0.78	0.79	0.78	76
Medium (31-60 min)	0.52	0.52	0.52	42
Long (61-120 min)	0.73	0.65	0.69	57
Very Long (>120 min)	0.86	0.90	0.88	94
accuracy			0.76	269
macro avg	0.72	0.72	0.72	269
weighted avg	0.76	0.76	0.76	269

As shown in the figure above, the model performed significantly better than the Linear Regression model as demonstrated by an overall accuracy score of 0.76. This difference establishes the model's ability to capture the nonlinear patterns that are found in the recipe dataset. The model performed best when predicting classes "Quick" and "Very Long", likely due to the prevalence of key words that inherently relate to time duration such as "quick" and "fast". Other strong indicators for duration such as cooking techniques like "marinate" are found in these classes. The "Medium" class having the lowest performance is as expected, as it typically has less distinctive language. It was noted that the model's performance improved greatly after implementing RecipeBERT's embeddings from an initial accuracy score of 0.67.

When tasking a large language model like ChatGPT (GPT 5.3) to predict explicit time and time range using the same corpus that was given to the Linear and Logistic Regression models, the LLM reached a bucket accuracy of 67% and MAE of 29 minutes. This classification accuracy reflects the initial results from the Logistic Regression model prior to implementation of RecipeBERT which is expected as ChatGPT lacks the dense embeddings that the Logistic Regression model has. ChatGPT's lack of task-specific training results in a limited ability to understand time based on semantics. With an MAE of 29 minutes, however, ChatGPT prevails over the Linear Regression model.

IV. Conclusions/Future Work

Each model is designed for a different task, however, the Logistic Regression model proved to be the most successful at accomplishing its goal of accurately predicting time ranges. Although LLMs like ChatGPT have weaker performance scores as a classification model without the context that the Logistic Regression model has, it still provides useful, though less accurate, time estimation that can be used as a baseline. Despite its low performance and unsuitability for the task, Linear Regression's usefulness came in the form of extreme confirmation that recipe time estimation is nonlinear.

We hope that our Logistic Regression model can be used to better gauge time ranges beyond what is listed on a recipe, offering everyday cooks an alternative method of time estimation. With a classification model, the hope is that with a time range as opposed to an explicit time prediction, fitting cooking into their busy schedules will not be so daunting of a task.

For future work, we'd like to apply data sampling techniques such as stratified sampling, class weighting, or SMOTE to minimize the difficulty that comes with properly representing each class. We'd also like to look at other models such as Neural Networks or Transformers for further experimentation and model comparison.

References

- D. Mereddy and J. S. R. Beedareddy, "Enabling Next-Generation Smart Homes Through Bert Personalized Food Recommendations - RecipeBERT," 2024 IEEE/WIC International Conference on Web Intelligence and Intelligent Agent Technology (WI-IAT), Bangkok, Thailand, 2024, pp. 796-803, doi: 10.1109/WI-IAT62293.2024.00130. keywords: {Adaptation models;Analytical models;Reviews;Semantics;Smart homes;Manuals;Transformers;Recommender systems;Next generation networking;Testing;Personalized food recommendations;BERT;SBERT;RecipeBERT},
- Monsivais, Pablo et al. "Time spent on home food preparation and indicators of healthy eating." American journal of preventive medicine vol. 47,6 (2014): 796-802. doi:10.1016/j.amepre.2014.07.033